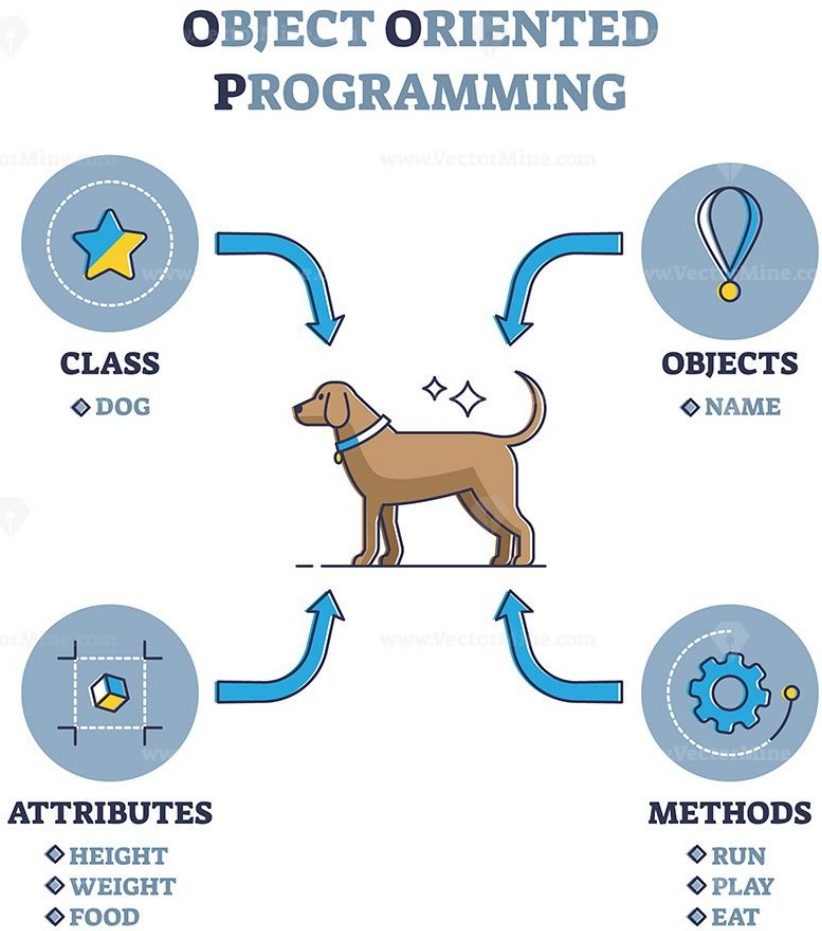
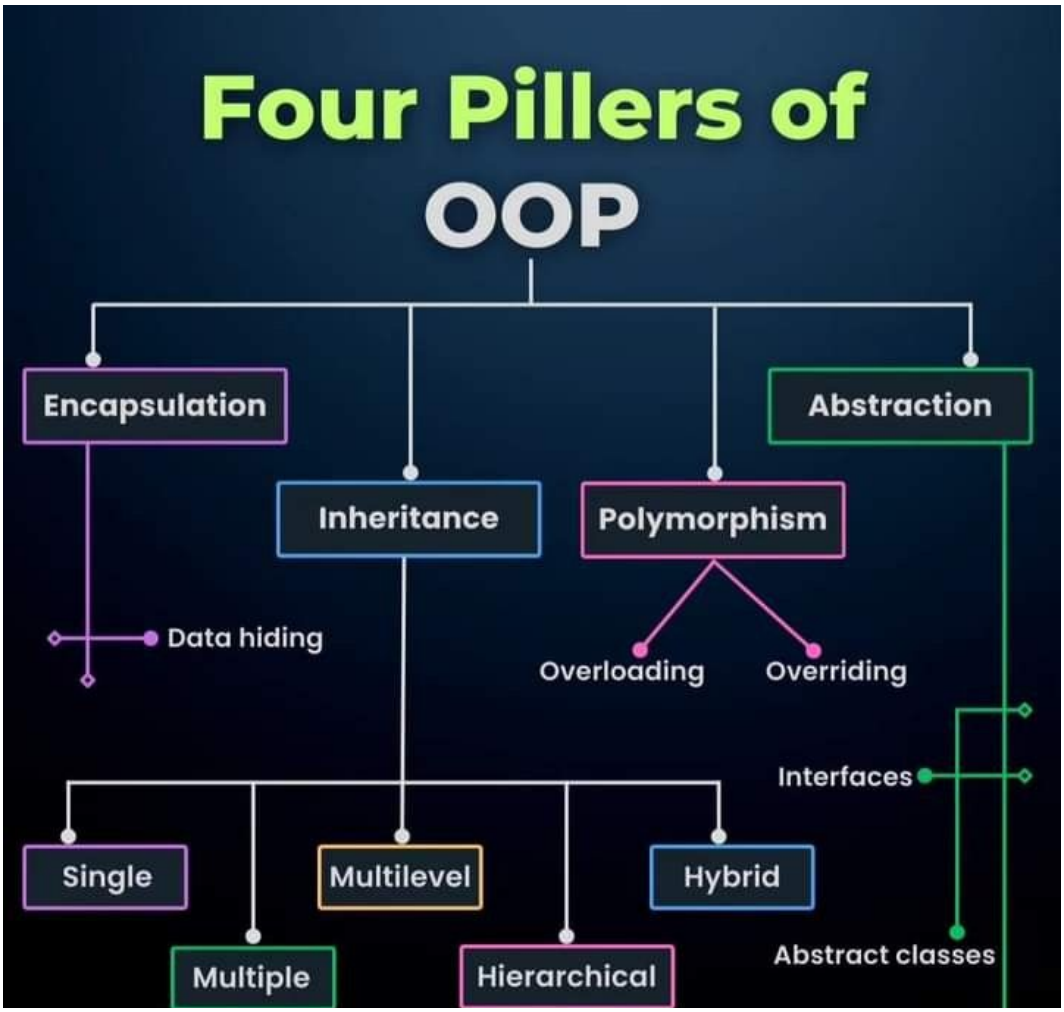
阿里云天池 Python 训练营 - Part4

20. 面向对象基础

20.1 类和对象



- 万物皆对象
- 类是具有相同属性和方法的多个对象的集合
- 属性是表示对象的描述性信息（身高、体重、年龄等），方法是表示对象的动作信息（学习、看电视、开车等）
- 将属性和方法包裹在一起的操作叫做封装
- 类和类之间可以有继承的关系，也就是基类和子类
- 父子类之间可以存在方法/函数的重载和重写





1. **封装（Encapsulation）**
    - 通过“数据隐藏（Data hiding）”保护类内部的状态和行为不被外界直接访问。
2. **继承（Inheritance）**
    - 展示了四种继承模式：
        - 单继承（Single）
        - 多继承（Multiple）
        - 多级继承（Multilevel）
        - 分层继承（Hierarchical）
        - 以及混合继承（Hybrid）
3. **多态（Polymorphism）**
    - 包括两种实现方式：
        - 方法重载（Overloading）
        - 方法重写（Overriding）
4. **抽象（Abstraction）**
    - 通过接口（Interfaces）和抽象类（Abstract classes）实现，隐藏实现细节，只提供必要的功能接口。

总结：

> 总结：

- **封装**提供了数据安全性，**继承**增强了代码重用性，**多态**提高了灵活性，**抽象**则简化了复杂性。
- 这些原则相辅相成，共同构成了面向对象编程的基础框架。

```
from abc import ABC, abstractmethod

# 抽象类 (接口)
class Animal(ABC):
    @abstractmethod
    def sound(self):
        """抽象方法，子类必须实现"""
        pass

    @abstractmethod
    def move(self):
        """抽象方法，子类必须实现"""
        pass

    # 普通方法
    def info(self):
        return "This is a generic animal."


# 父子类的继承与重写
class Dog(Animal):
    def sound(self):
        return "Woof! Woof!"  # 重写抽象方法
```

```
    def move(self):
        return "The dog runs."

    # 重写父类的方法
    def info(self):
        return "This is a dog, a loyal
animal."

    # 方法重载：通过默认参数实现
    def fetch(self, item=None):
        if item:
            return f"The dog fetches
the {item}."
        else:
            return "The dog fetches
nothing."


class Cat(Animal):
    def sound(self):
        return "Meow! Meow!"  # 重写抽象
方法

    def move(self):
        return "The cat jumps."

    # 类内方法重载：通过参数个数实现
    def scratch(self, target=None):
        if target:
            return f"The cat scratches
the {target}!"
        return "The cat scratches
```

```
randomly."


# 多态函数
def animal_actions(animal: Animal):
    print(f"Sound: {animal.sound()}")
    print(f"Move: {animal.move()}")
    print(f"Info: {animal.info()}")


# 创建实例
dog = Dog()
cat = Cat()

# 多态调用
print("Dog actions:")
animal_actions(dog)
print(dog.fetch("ball"))  # 重载方法调用
print(dog.fetch())        # 使用默认参数调
用

print("\nCat actions:")
animal_actions(cat)
print(cat.scratch("couch"))  # 重载方法调
用
print(cat.scratch())         # 使用无参调
用
```

说明

- 继承与重写（Overriding）：
    - 在 Dog 类中，方法 info 重写了 Animal 类的

info 方法，为父类方法提供了特定子类的实现。
    - 同样，sound 和 move 方法实现了抽象类中的定义。

- 方法重载（Overloading）：
    - 通过默认参数：Dog 类中的 fetch 方法使用默认参数来实现重载，允许调用时传递参数或不传递参数。
    - 通过参数个数：Cat 类中的 scratch 方法通过检查传入的参数个数和类型，实现不同调用方式的行为。

- 多态：

* 函数 animal_actions 调用不同子类的实现，通过统一接口操作不同类型的对象。

20.2 定义类

在 Python 中可以使用 `class` 关键字定义类，然后在类中通过之前学习过的函数来定义方法，这样就可以将对象的动态特征描述出来，代码如下所示。

```
class Student(object):

    # __init__ 是一个特殊方法用于在创建对象
时进行初始化操作
    # 通过这个方法我们可以为学生对象绑定 name
和 age 两个属性
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def study(self, course_name):
        print('%s 正在学习 %s.' %
(self.name, course_name))

    # PEP 8 要求标识符的名字用全小写多个单词
用下划线连接
    # 但是部分程序员和公司更倾向于使用驼峰命名
```

```
法(驼峰标识)
    def watch_movie(self):
        if self.age < 18:
            print('%s 只能观看《熊出没》.'
% self.name)
        else:
            print('%s 正在观看美国动作大电
影.' % self.name)
```

> **说明：** 写在类中的函数，我们通常称之为（对象的）方法，这些方法就是对象可以接收的消息。

20.3 创建和使用对象

当我们定义好一个类之后，可以通过下面的方式来创建对象并给对象发消息。

```
def main():
```

```
    # 创建学生对象并指定姓名和年龄
    stu1 = Student('林克', 38)
    # 给对象发 study 消息
    stu1.study('Python 程序设计')
    # 给对象发 watch_av 消息
    stu1.watch_movie()stu2 = Student('王
大锤', 15)
    stu2.study('思想品德')
    stu2.watch_movie()

if __name__ == '__main__':
    main()
```

20.4 访问可见性问题

对于上面的代码，有 C++、Java、C# 等编程经验的程序员可能会问，我们给 `Student` 对象绑定的 `name` 和 `age` 属性到底具有怎样的访问权限（也称为可见性）。

在 Java 中，访问级别决定了类、方法、变量和构造函数的可访问范围。以下是 Java 中 **访问修饰符（Access Modifiers）** 的级别表格：

修饰符	类内	同包	子类（跨包）	其他包
public	✅	✅	✅	✅

protected	✅	✅	✅	❌
default *(无修饰符)*	✅	✅	❌	❌
private	✅	❌	❌	❌

说明：

1. **public**：
    - 任何地方都可以访问，包括跨包。
    - 常用于对外开放的接口或方法。
2. **protected**：
    - 在同一个包内可以访问，跨包时仅限子类可访问。
    - 适合用于提供子类扩展功能但不完全对外暴露的内容。
3. **default**（包级访问）：
    - 如果没有显式修饰符，默认为包级访问权限。
    - 仅在同一个包内可以访问，跨包无法访问。
4. **private**：
    - 仅限于类内部访问。
    - 常用于封装数据和实现细节。

20.4.1 Java 可访问性示例代码

```
package example;

public class AccessModifiersExample {

    public String publicField = "Public
Field";
    protected String protectedField =
"Protected Field";
    String defaultField = "Default
Field";
    private String privateField =
"Private Field";

    public void publicMethod() {
        System.out.println("Public
Method");
    }

    protected void protectedMethod() {
        System.out.println("Protected
Method");
    }

    void defaultMethod() {
        System.out.println("Default
Method");

    }
```

```
    private void privateMethod() {
        System.out.println("Private
Method");
    }

    public void showAccess() {
        // 类内可以访问所有修饰符

System.out.println(publicField);

System.out.println(protectedField);

System.out.println(defaultField);

System.out.println(privateField);
        publicMethod();
        protectedMethod();
        defaultMethod();
        privateMethod();
    }
}
```

20.4.2 Java 可访问性示例访问（跨包）

```
package another;

import example.AccessModifiersExample;

public class TestAccess {
    public static void main(String[]
args) {
        AccessModifiersExample obj =
new AccessModifiersExample();


System.out.println(obj.publicField);
// ✅ 可访问
        //
System.out.println(obj.protectedField);
// ❌ 不可访问（跨包且非子类）
        //
System.out.println(obj.defaultField);
```

```
// ❌ 不可访问（跨包）
        //
System.out.println(obj.privateField);
// ❌ 不可访问（仅类内）

        obj.publicMethod();  // ✅ 可访
问
        // obj.protectedMethod();  //
❌ 不可访问（跨包且非子类）
        // obj.defaultMethod();    //
❌ 不可访问（跨包）
        // obj.privateMethod();    //
❌ 不可访问（仅类内）
    }
}
```

通过表格和代码示例，可以直观了解 Java 中访问级别的具体应用和作用范围！

因为在很多面向对象编程语言中，我们通常会将对象的属性设置为私有的（private）或受保护的（protected），简单的说就是不允许外界访问，而对象的方法通常都是公开的（public），因为公开的方法就是对象能够接受的消息。

20.4.3 Python 中的访问级别

在 Python 中，属性和方法的访问权限只有两种，也就是公开的和私有的，如果希望属性是私有的，在给属性命名时可以用两个下划线作为开头，下面的代码可

以验证这一点。

```
class Test:

    def __init__(self, foo):
        self.__foo = foo

    def __bar(self):
        print(self.__foo)
        print('__bar')

def main():
    test = Test('hello')
    # AttributeError: 'Test' object has
no attribute '__bar'
    test.__bar()
    # AttributeError: 'Test' object has
no attribute '__foo'
    print(test.__foo)

if __name__ == "__main__":
    main()
```

但是，Python 并没有从语法上严格保证私有属性或方法的私密性，它只是给私有的属性和方法换了一个名字来妨碍对它们的访问，事实上如果你知道更换名字的规则仍然可以访问到它们，下面的代码就可以验证这一点。之所以这样设定，可以用这样一句名言加以解释，就是 "**We are all consenting adults here**"。因为绝大多数程序员都认为开放比封闭要好，而且程序员要自己为自己的行为负责。

```
class Test:

    def __init__(self, foo):
        self.__foo = foo

    def __bar(self):
        print(self.__foo)
        print('__bar')

def main():
    test = Test('hello')

test._Test__bar()print(test._Test__foo)

if __name__ == "__main__":
    main()
```

在实际开发中，我们并不建议将属性设置为私有的，

因为这会导致子类无法访问（后面会讲到）。

所以大多数 Python 程序员会遵循一种命名惯例就是让属性名以单下划线开头来表示属性是受保护的，本类之外的代码在访问这样的属性时应该要保持慎重。这种做法并不是语法上的规则，单下划线开头的属性和方法外界仍然是可以访问的，所以更多的时候它是一种暗示或隐喻。

20.5 面向对象的支柱

面向对象有三大支柱（算上抽象是 4 个支柱）： **封装、继承和多态。**

后面两个概念在下一个章节中进行详细的说明，这里我们先说一下什么是封装。

我们在类中定义的方法其实就是把数据和对数据的操作封装起来了，在我们创建了对象之后，只需要给对象发送一个消息（调用方法）就可以执行方法中的代码，也就是说我们只需要知道方法的名字和传入的参数（方法的外部视图），而不需要知道方法内部的实现细节（方法的内部视图）。

20.6 练习

练习 1：定义一个类描述数字时钟。

```
```

```
from time import sleep

class Clock(object):
    """ 数字时钟 """

    def __init__(self, hour=0,
minute=0, second=0):
        """
        初始化方法
        :param hour: 时
        :param minute: 分
        :param second: 秒
        """
        self._hour = hour
        self._minute = minute
        self._second = second

    def run(self):
        """ 走字 """
        self._second += 1
        if self._second == 60:
            self._second = 0
            self._minute += 1
            if self._minute == 60:
                self._minute = 0
                self._hour += 1
                if self._hour == 24:
                    self._hour = 0

    def show(self):
        """ 显示时间 """
        return '%02d:%02d:%02d' % \
               (self._hour,
```

```
self._minute, self._second)


def main():
    clock = Clock(23, 59, 58)
    while True:
        print(clock.show())
        sleep(1)
        clock.run()

if __name__ == '__main__':
    main()
```

练习 2：定义一个类描述平面上的点并提供移动点和计算到另一个点距离的方法。

```
from math import sqrt

class Point(object):

    def __init__(self, x=0, y=0):
        """
        初始化方法
        :param x: 横坐标
        :param y: 纵坐标
        """
        self.x = x
        self.y = y

    def move_to(self, x, y):
        """
        移动到指定位置
```

```
        移动到指定位置
        :param x: 新的横坐标
        "param y: 新的纵坐标"""
        self.x = x
        self.y = y

    def move_by(self, dx, dy):
        """
        移动指定的增量
        :param dx: 横坐标的增量
        "param dy: 纵坐标的增量"""
        self.x += dx
        self.y += dy

    def distance_to(self, other):
        """
        计算与另一个点的距离
        :param other: 另一个点
        """
        dx = self.x - other.x
        dy = self.y - other.y
        return sqrt(dx ** 2 + dy ** 2)

    def __str__(self):
        return '(%s, %s)' %
(str(self.x), str(self.y))

def main():
    p1 = Point(3, 5)
    p2 = Point()print(p1)
    print(p2)
    p2.move_by(-1, 2)
    print(p2)
```

```
    print(p1.distance_to(p2))

if __name__ == '__main__':
    main()
```

21. 面向对象进阶

在前面的章节我们已经了解了面向对象的入门知识，知道了如何定义类，如何创建对象以及如何给对象发消息。为了能够更好的使用面向对象编程思想进行程序开发，我们还需要对Python中的面向对象编程进行更为深入的了解。

21.1 @property装饰器

之前我们讨论过Python中属性和方法访问权限的问题，虽然我们不建议将属性设置为私有的，但是如果直接将属性暴露给外界也是有问题的，比如我们没有办法检查赋给属性的值是否有效。

我们之前的建议是将属性命名以单下划线开头，通过这种方式来暗示属性是受保护的，不建议外界直接访问，那么如果想访问属性可以通过属性的 getter（访问器）和 setter（修改器）方法进行对应的操作。

如果要做到这点，就可以考虑使用@property包装器来包装 getter 和 setter 方法，使得对属性的访问既安全又方便，代码如下所示。

```
class Person(object):

    def __init__(self, name, age):
        self._name = name
        self._age = age

    # 访问器 - getter方法
    @property
    def name(self):
        return self._name

    # 访问器 - getter方法
    @property
    def age(self):
        return self._age

    # 修改器 - setter方法
    @age.setter
    def age(self, age):
        self._age = age
```

```
        self._age = age

    def play(self):
        if self._age <= 16:
            print('%s正在玩飞行棋.' %
self._name)
        else:
            print('%s正在玩斗地主.' %
self._name)


def main():
    person = Person('王大锤', 12)
    person.play()
    person.age = 22
    person.play()
    # person.name = '白元芳'  #
AttributeError: can't set attribute

if __name__ == '__main__':
    main()
```

21.2 slots魔法

Python 是一门动态语言，通常动态语言允许我们在程序运行时给对象绑定新的属性或方法，当然也可以对已经绑定的属性和方法进行解绑定。但是如果我们需要限定自定义类型的对象只能绑定某些属性，可以通过在类中定义**slots**变量来进行限定。

需要注意的是slots的限定只对当前类的对象生效，对子类并不起任何作用

了类外部的作用。

```
class Person(object):

    # 限定Person对象只能绑定_name, _age和
_gender属性
    __slots__ = ('_name', '_age',
'_gender')

    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        return self._name

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, age):
        self._age = age

    def play(self):
        if self._age <= 16:
            print('%s正在玩飞行棋.' %
self._name)
        else:
            print('%s正在玩斗地主.' %
self._name)
```

```
def main():
    person = Person('王大锤', 22)
    person.play()
    person._gender = '男'
    # AttributeError: 'Person' object
has no attribute '_is_gay'
    # person._is_gay = True
```

21.3 静态方法和类方法

之前，我们在类中定义的方法都是对象方法，也就是说这些方法都是发送给对象的消息。

实际上，我们写在类中的方法并不需要都是对象方法，例如我们定义一个“三角形”类，通过传入三条边长来构造三角形，并提供计算周长和面积的方法，但是传入的三条边长未必能构造出三角形对象，因此我们可以先写一个方法来验证三条边长是否可以构成三角形，这个方法很显然就不是对象方法，因为在调用这个方法时三角形对象尚未创建出来（因为都不知道三条边能不能构成三角形），所以这个方法是属于三角形类而并不属于三角形对象的。

我们可以使用静态方法来解决这类问题，代码如下所示。

```
from math import sqrt

class Triangle(object):
```

```
    def __init__(self, a, b, c):
        self._a = a
        self._b = b
        self._c = c

    @staticmethod
    def is_valid(a, b, c):
        return a + b > c and b + c > a
and a + c > b

    def perimeter(self):
        return self._a + self._b +
self._c

    def area(self):
        half = self.perimeter() / 2
        return sqrt(half * (half -
self._a) *
                    (half - self._b) *
(half - self._c))


def main():
    a, b, c = 3, 4, 5
    # 静态方法和类方法都是通过给类发消息来调用
的
    if Triangle.is_valid(a, b, c):
        t = Triangle(a, b, c)
        print(t.perimeter())
        # 也可以通过给类发消息来调用对象方法
但是要传入接收消息的对象作为参数
        # print(Triangle.perimeter(t))
        print(t.area())
```

```
            print(t.area())
            # print(Triangle.area(t))
        else:
            print('无法构成三角形.')

if __name__ == '__main__':
    main()
```

和静态方法比较类似，Python还可以在类中定义类方法，类方法的第一个参数约定名为 cls，它代表的是当前类相关的信息的对象（类本身也是一个对象，有的

地方也称之为类的元数据对象），通过这个参数我们可以获取和类相关的信息并且可以创建出类的对象，代码如下所示。

```
from time import time, localtime, sleep

class Clock(object):
    """数字时钟"""
```

```
    def __init__(self, hour=0,
minute=0, second=0):
        self._hour = hour
        self._minute = minute
        self._second = second

    @classmethod
    def now(cls):
        ctime = localtime(time())
        return cls(ctime.tm_hour,
ctime.tm_min, ctime.tm_sec)

    def run(self):
        """走字"""
        self._second += 1
        if self._second == 60:
            self._second = 0
            self._minute += 1
            if self._minute == 60:
                self._minute = 0
                self._hour += 1
                if self._hour == 24:
                    self._hour = 0

    def show(self):
        """显示时间"""
        return '%02d:%02d:%02d' % \
               (self._hour,
self._minute, self._second)


def main():
```

```
    # 通过类方法创建对象并获取系统时间
    clock = Clock.now()
    while True:
        print(clock.show())
        sleep(1)
        clock.run()

if __name__ == '__main__':
    main()
```

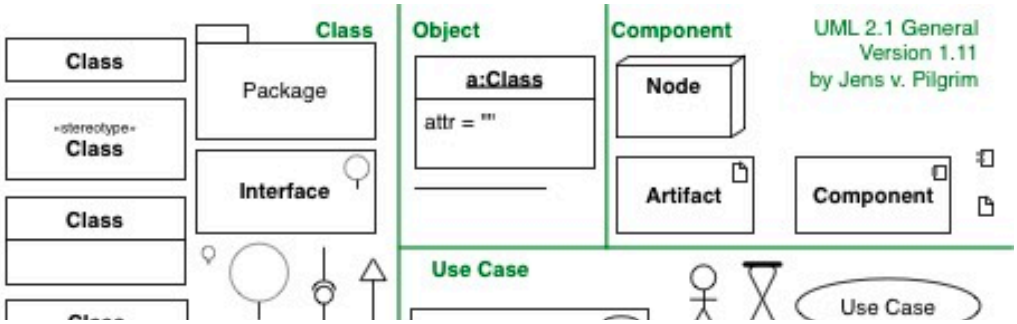
21.4 类之间的关系

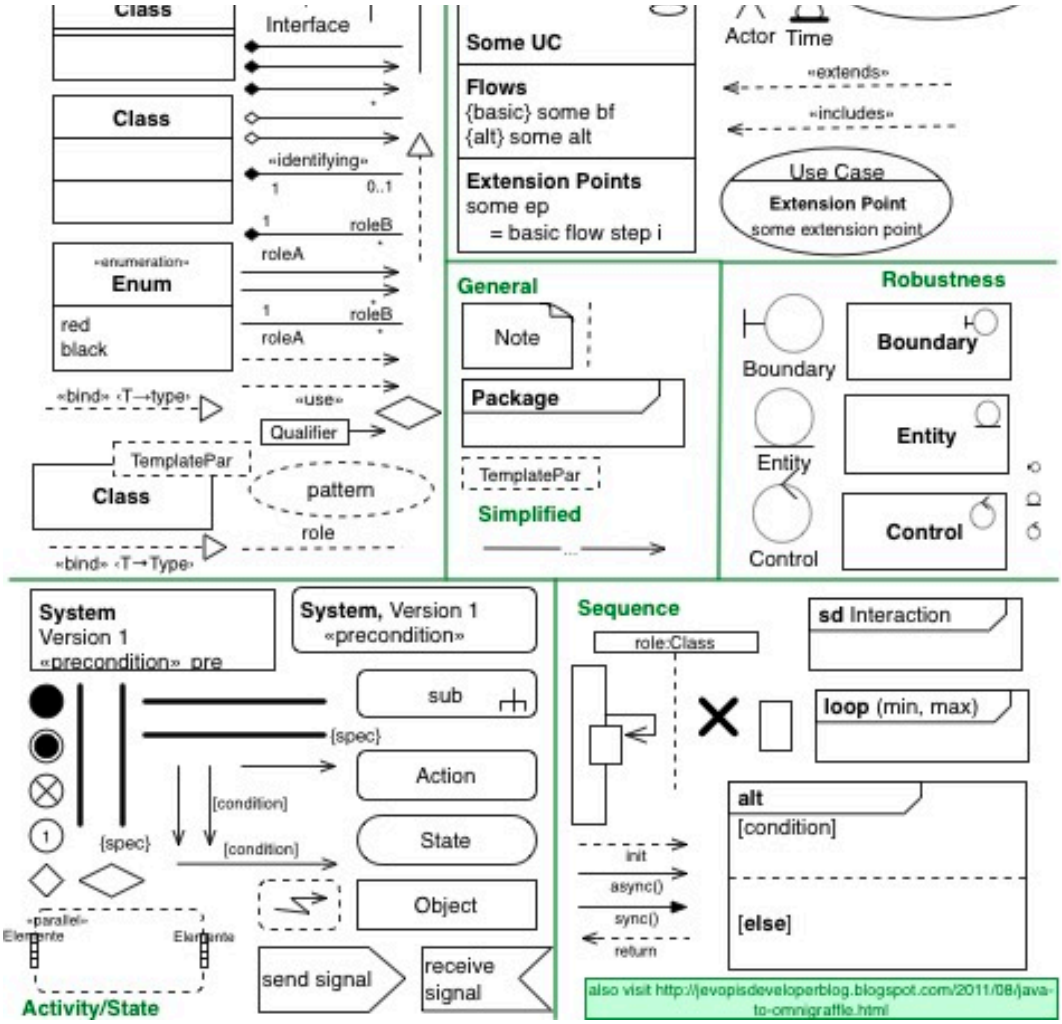
简单的说，类和类之间的关系有三种：is-a、has-a 和 use-a 关系。

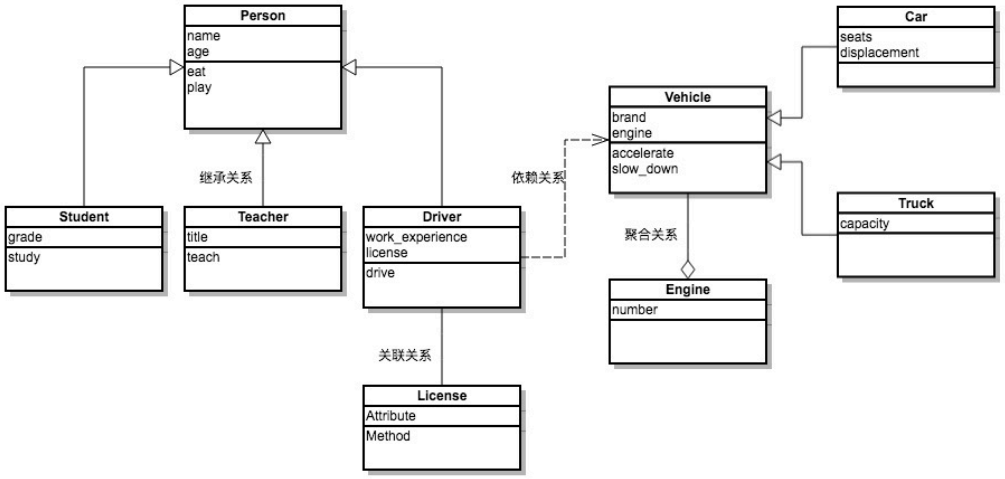
- is-a 关系也叫继承或泛化，比如学生和人的关系、手机和电子产品的关系都属于继承关系。
- has-a 关系通常称之为关联，比如部门和员工的关系，汽车和引擎的关系都属于关联关系；关联关系如果是整体和部分的关联，那么我们称之为聚合关系；如果整体进一步负责了部分的生命周期（整体和部分是不可分割的，同时同在也同时消亡），那么这种就是最强的关联关系，我们称之为合成关系。
- use-a 关系通常称之为依赖，比如司机有一个驾驶的行为（方法），其中（的参数）使用到了汽车，那么司机和汽车的关系就是依赖关系。

我们可以使用一种叫做UML（统一建模语言）的东西

来进行面向对象建模，其中一项重要的工作就是把类和类之间的关系用标准化的图形符号描述出来。关于UML我们在这里不做详细的介绍，有兴趣的读者可以自行阅读《UML面向对象设计基础》一书。







利用类之间的这些关系，我们可以在已有类的基础上来完成某些操作，也可以在已有类的基础上创建新的类，这些都是实现代码复用的重要手段。复用现有的代码不仅可以减少开发的工作量，也有利于代码的管

理和维护，这是我们在日常工作中都会使用到的技术手段。

21.5 继承和多态

刚才我们提到了，可以在已有类的基础上创建新类，这其中的一种做法就是让一个类从另一个类那里将属性和方法直接继承下来，从而减少重复代码的编写。提供继承信息的我们称之为父类，也叫超类或基类；

得到继承信息的我们称之为子类，也叫派生类或衍生类。

子类除了继承父类提供的属性和方法，还可以定义自己特有的属性和方法，所以子类比父类拥有的更多的能力，在实际开发中，我们经常会用子类对象去替换掉一个父类对象，这是面向对象编程中一个常见的行为，对应的原则称之为里氏替换原则。

下面我们先看一个继承的例子。

```
class Person(object):
    """人"""

    def __init__(self, name, age):
        self._name = name
        self._age = age

    @property
    def name(self):
        return self._name
```

```
    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, age):
        self._age = age

    def play(self):
        print('%s正在愉快的玩耍.' %
self._name)

    def watch_av(self):
        if self._age >= 18:
            print('%s正在观看爱情动作片.'
% self._name)
        else:
            print('%s只能观看《熊出没》.'
% self._name)


class Student(Person):
    """学生"""

    def __init__(self, name, age,

grade):
        super().__init__(name, age)
        self._grade = grade

    @property
    def grade(self):
```

```
    def grade(self):
        return self._grade

    @grade.setter
    def grade(self, grade):
        self._grade = grade

    def study(self, course):
        print('%s的%s正在学习%s.' %
(self._grade, self._name, course))


class Teacher(Person):
    """老师"""

    def __init__(self, name, age,
title):
        super().__init__(name, age)
        self._title = title

    @property
    def title(self):
        return self._title

    @title.setter
    def title(self, title):
        self._title = title

    def teach(self, course):
        print('%s%s正在讲%s.' %
(self._name, self._title, course))


def main():
```

```
def main():
    stu = Student('王大锤', 15, '初三')
    stu.study('数学')
    stu.watch_av()
    t = Teacher('林克', 38, '砖家')
    t.teach('Python程序设计')
    t.watch_av()


if __name__ == '__main__':

    main()
```

子类在继承了父类的方法后，可以对父类已有的方法给出新的实现版本，这个动作称之为方法重写（override）。

通过方法重写我们可以让父类的同一个行为在子类中拥有不同的实现版本，当我们调用这个经过子类重写的方法时，不同的子类对象会表现出不同的行为，这个就是多态（poly-morphism）。

```
from abc import ABCMeta, abstractmethod


class Pet(object, metaclass=ABCMeta):
    """宠物"""
```

```
    def __init__(self, nickname):
        self._nickname = nickname

    @abstractmethod
    def make_voice(self):
        """发出声音"""
        pass


class Dog(Pet):
    """狗"""

    def make_voice(self):
        print('%s: 汪汪汪...' %
self._nickname)


class Cat(Pet):
    """猫"""

    def make_voice(self):
        print('%s: 喵...喵...' %
self._nickname)


def main():
    pets = [Dog('旺财'), Cat('凯蒂'),
Dog('大黄')]
    for pet in pets:
        pet.make_voice()


if __name__ == '__main__':
```

```
if __name__ == '__main__':
    main()
```

在上面的代码中，我们将 `Pet` 类处理成了一个抽象类，所谓抽象类就是不能够创建对象的类，这种类的存在就是专门为了让其他类去继承它。Python从语法层面并没有像Java或C#那样提供对抽象类的支持，但是我们可以通过 `abc` 模块的 `ABCMeta` 元类和 `abstractmethod` 包装器来达到抽象类的效果，如果一个类中存在抽象方法那么这个类就不能够实例化（创建对象）。上面的代码中， `Dog` 和 `Cat` 两个子类分别对 `Pet` 类中的 `make_voice` 抽象方法进行了重写并给出了不同的实现版本，当我们在 `main` 函数中调用该方法时，这个方法就表现出了多态行为（同样的方法做了不同的事情）。

22. 综合案例

案例1：奥特曼打小怪兽。

```
from abc import ABCMeta, abstractmethod
from random import randint, randrange

class Fighter(object,
metaclass=ABCMeta):
    """战斗者"""

    # 通过__slots__魔法限定对象可以绑定的成
员变量
```

```
    __slots__ = ('_name', '_hp')

    def __init__(self, name, hp):
        """
        初始化方法
        :param name: 名字
        :param hp: 生命值
        """
        self._name = name
        self._hp = hp

    @property
    def name(self):
        return self._name

    @property
    def hp(self):
        return self._hp

    @hp.setter
    def hp(self, hp):
        self._hp = hp if hp >= 0 else 0

    @property
    def alive(self):
        return self._hp > 0

    @abstractmethod
    def attack(self, other):
        """
        攻击
        :param other: 被攻击的对象
        """
```

```
        pass


class Ultraman(Fighter):
    """奥特曼"""

    __slots__ = ('_name', '_hp', '_mp')

    def __init__(self, name, hp, mp):
        """
        初始化方法
        :param name: 名字
        :param hp: 生命值
        :param mp: 魔法值
        """
        super().__init__(name, hp)
        self._mp = mp

    def attack(self, other):
        other.hp -= randint(15, 25)

    def huge_attack(self, other):
        """
        究极必杀技(打掉对方至少50点或四分之三
的血)
        :param other: 被攻击的对象
        :return: 使用成功返回True否则返回
False
        """
        if self._mp >= 50:
            self._mp -= 50
            injury = other.hp * 3 // 4
```

```
            injury = injury if injury
>= 50 else 50
            other.hp -= injury
            return True
        else:
            self.attack(other)
            return False

    def magic_attack(self, others):

        """
        魔法攻击
        :param others: 被攻击的群体
        :return: 使用魔法成功返回True否则返
回False
        """
        if self._mp >= 20:
            self._mp -= 20
            for temp in others:
                if temp.alive:
                    temp.hp -=
randint(10, 15)
            return True
        else:
            return False

    def resume(self):
        """恢复魔法值"""
        incr_point = randint(1, 10)
        self._mp += incr_point
        return incr_point

    def __str__(self):
```

```
        return '~~~%s奥特曼~~~\n' %
self._name + \
            '生命值: %d\n' % self._hp +
\
            '魔法值: %d\n' % self._mp


class Monster(Fighter):
    """小怪兽"""

    __slots__ = ('_name', '_hp')

    def attack(self, other):
        other.hp -= randint(10, 20)

    def __str__(self):
        return '~~~%s小怪兽~~~\n' %
self._name + \
            '生命值: %d\n' % self._hp


def is_any_alive(monsters):
    """判断有没有小怪兽是活着的"""
    for monster in monsters:
        if monster.alive > 0:

            return True
    return False


def select_alive_one(monsters):
    """选中一只活着的小怪兽"""
    monsters_len = len(monsters)
```

```
    while True:
        index = randrange(monsters_len)
        monster = monsters[index]
        if monster.alive > 0:
            return monster


def display_info(ultraman, monsters):
    """显示奥特曼和小怪兽的信息"""
    print(ultraman)
    for monster in monsters:
        print(monster, end='')


def main():
    u = Ultraman('林克', 1000, 120)
    m1 = Monster('狄仁杰', 250)
    m2 = Monster('白元芳', 500)
    m3 = Monster('王大锤', 750)
    ms = [m1, m2, m3]
    fight_round = 1
    while u.alive and is_any_alive(ms):
        print('========第%02d回合
========' % fight_round)
        m = select_alive_one(ms)  # 选中
一只小怪兽
        skill = randint(1, 10)   # 通过
随机数选择使用哪种技能
        if skill <= 6:  # 60%的概率使用普
通攻击
            print('%s使用普通攻击打了%s.'
% (u.name, m.name))
            u.attack(m)
```

```
            print('%s的魔法值恢复了%d点.'
% (u.name, u.resume()))
        elif skill <= 9:  # 30%的概率使用
魔法攻击(可能因魔法值不足而失败)
            if u.magic_attack(ms):
                print('%s使用了魔法攻击.'
% u.name)
            else:

                print('%s使用魔法失败.' %
u.name)
        else:  # 10%的概率使用究极必杀技(如
果魔法值不足则使用普通攻击)
            if u.huge_attack(m):
                print('%s使用究极必杀技虐
了%s.' % (u.name, m.name))
            else:
                print('%s使用普通攻击打
了%s.' % (u.name, m.name))
                print('%s的魔法值恢复了%d
点.' % (u.name, u.resume()))
        if m.alive > 0:  # 如果选中的小怪
兽没有死就回击奥特曼
            print('%s回击了%s.' %
(m.name, u.name))
            m.attack(u)
        display_info(u, ms)  # 每个回合结
束后显示奥特曼和小怪兽的信息
        fight_round += 1
    print('\n========战斗结
束!========\n')
    if u.alive > 0:
        print('%s奥特曼胜利!' % u.name)
```

```
        print('%s英雄获胜!' % hero.name)
    else:
        print('小怪兽胜利!')


if __name__ == '__main__':
    main()
```

案例2：扑克游戏。

```
import random

class Card(object):
    """一张牌"""

    def __init__(self, suite, face):
        self._suite = suite
        self._face = face

    @property
    def face(self):
        return self._face

    @property

    def suite(self):
        return self._suite

    def __str__(self):
        if self._face == 1:
            face_str = 'A'
        elif self._face == 11:
            face_str = 'J'
```

```
            face_str = 'J'
        elif self._face == 12:
            face_str = 'Q'
        elif self._face == 13:
            face_str = 'K'
        else:
            face_str = str(self._face)
        return '%s%s' % (self._suite,
face_str)

    def __repr__(self):
        return self.__str__()


class Poker(object):
    """一副牌"""

    def __init__(self):
        self._cards = [Card(suite,
face)  for suite in '♠♥♣♦' for face in
range(1, 14)]
        self._current = 0

    @property
    def cards(self):
        return self._cards

    def shuffle(self):
        """洗牌(随机乱序)"""
        self._current = 0
        random.shuffle(self._cards)

    @property
```

```
    def next(self):
        """发牌"""
        card =
self._cards[self._current]
        self._current += 1
        return card

    @property

    def has_next(self):
        """还有没有牌"""
        return self._current <
len(self._cards)


class Player(object):
    """玩家"""

    def __init__(self, name):
        self._name = name
        self._cards_on_hand = []

    @property
    def name(self):
        return self._name

    @property
    def cards_on_hand(self):
        return self._cards_on_hand

    def get(self, card):
        """摸牌"""
```

```
self._cards_on_hand.append(card)

    def arrange(self, card_key):
        """玩家整理手上的牌"""

self._cards_on_hand.sort(key=card_key)


# 排序规则-先根据花色再根据点数排序
def get_key(card):
    return (card.suite, card.face)


def main():
    p = Poker()
    p.shuffle()
    players = [Player('东邪'),
Player('西毒'), Player('南帝'),
Player('北丐')]
    for _ in range(13):
        for player in players:
            player.get(p.next)
    for player in players:
        print(player.name + ':', end='

')
        player.arrange(get_key)
        print(player.cards_on_hand)


if __name__ == '__main__':
    main()
```

> **说明：** 大家可以自己尝试在上面代码的基础上写一个简单的扑克游戏，例如21点(Black Jack)，游戏的规则可以自己在网上找一找。

案例3：工资结算系统。

```
"""
某公司有三种类型的员工 分别是部门经理、程序员和销售员
需要设计一个工资结算系统 根据提供的员工信息来计算月薪
部门经理的月薪是每月固定15000元
程序员的月薪按本月工作时间计算 每小时150元
销售员的月薪是1200元的底薪加上销售额5%的提成
"""
from abc import ABCMeta, abstractmethod


class Employee(object,
metaclass=ABCMeta):
    """员工"""

    def __init__(self, name):
        """
        初始化方法

        :param name: 姓名
        """
        self._name = name

    @property
    def name(self):
```

```
    def name(self):
        return self._name

    @abstractmethod
    def get_salary(self):
        """
        获得月薪


        :return: 月薪
        """
        pass


class Manager(Employee):
    """部门经理"""

    def get_salary(self):
        return 15000.0


class Programmer(Employee):
    """程序员"""

    def __init__(self, name,
working_hour=0):
        super().__init__(name)
        self._working_hour =
working_hour

    @property
    def working_hour(self):
        return self._working_hour
```

```
    @working_hour.setter
    def working_hour(self,
working_hour):
        self._working_hour =
working_hour if working_hour > 0 else 0

    def get_salary(self):
        return 150.0 *
self._working_hour


class Salesman(Employee):
    """销售员"""

    def __init__(self, name, sales=0):
        super().__init__(name)
        self._sales = sales

    @property
    def sales(self):
        return self._sales

    @sales.setter

    def sales(self, sales):
        self._sales = sales if sales >
0 else 0

    def get_salary(self):
        return 1200.0 + self._sales *
0.05
```

```
def main():
    emps = [
        Manager('刘备'), Programmer('诸葛
亮'),
        Manager('曹操'), Salesman('荀
彧'),
        Salesman('吕布'), Programmer('张
辽'),
        Programmer('赵云')
    ]
    for emp in emps:
        if isinstance(emp, Programmer):
            emp.working_hour =
int(input('请输入%s本月工作时间: ' %
emp.name))
        elif isinstance(emp, Salesman):
            emp.sales = float(input('请
输入%s本月销售额: ' % emp.name))
        # 同样是接收get_salary这个消息但是
不同的员工表现出了不同的行为(多态)
        print('%s本月工资为: ￥%s元' %
              (emp.name,
emp.get_salary()))


if __name__ == '__main__':
    main()
```